# Modern App Dev 1 Project Report

1.**Student Details:**

**Name:** Goteti Sai Anish
**Roll Number:** 25f2003342
**Email:** 25f2003342
**About Me:** I am a Diploma Level Student at IIT Madras BS Degree program with a deep interest in Web application development and Data Science concepts. I enjoy building meaningful applications that combine learning, analytics, and user experience. I enjoy creating various styles and designs for the dashboards in the application, and producing different options within the app to enhance the user's experience.

## 2. Project Details:

**Project Title: Trekking Management Application**

**Problem Statement:**

Adventure organizations need efficient systems to manage trekking activities that involve trek organizers, staff, and participants. Currently, many trekking groups rely on spreadsheets, phone calls, and manual coordination. This approach makes it challenging to manage trek approvals, track bookings, prevent overbooking, and maintain a history of treks..
I developed a Trekking Management Application, which is a web application that enables Admin, Trek Staff, and Users (Trekkers) to interact with the system according to their roles..

**Approach:**

The application was developed using Flask with a modular Blueprint-based architecture. It provides role-based functionality for Admin, Trek Staff, and Trekker/User, including authentication, trek management, staff assignment, user management, trek bookings, cancellation, and brochure management. SQLAlchemy is used for database operations and Flask-Login for authentication and authorization. REST-style APIs are provided for treks, users, and bookings, while Chart.js is used to visualize trek, booking, participant, and user-related statistics. The frontend uses HTML, CSS, JavaScript, and Jinja2 templates with role-specific dashboards.

## 3. AI/LLM Declaration:

ChatGPT 5.6 tools were used for approximately 15–20% of the project work. Assistance was mainly used for debugging and correcting code errors, improving code structure, refining text, and generating background images. The core application design, writing, implementation, integration, testing, and final decisions were carried out by me.

## 4. Technologies and Frameworks Used:

| Technology / Library | Purpose |
|---|---|
| **Flask** | Core backend web framework for application logic |
| **SQLAlchemy** | Object Relational Mapper for SQLite database |
| **Jinja2** | Template engine for rendering dynamic HTML pages |

| | |
|---|---|
| **Bootstrap 5** | Frontend styling and responsive design |
| **Chart.js** | To visualize trends such as activity vs. time |
| **Flask-Login** | User authentication and session management |
| **WTForms** | Frontend form validation |
| **SQLite** | Lightweight local database for storing user data |
| **JavaScript** | For user and staff-based interaction with the web page |
| **HTML** | For tables, forms and frontend structures |
| **CSS** | For styling, layouts, background and designing |

**5. Database Schema / ER Diagram:**

**Tables:**

User — stores user account and profile details such as ID, name, and email.

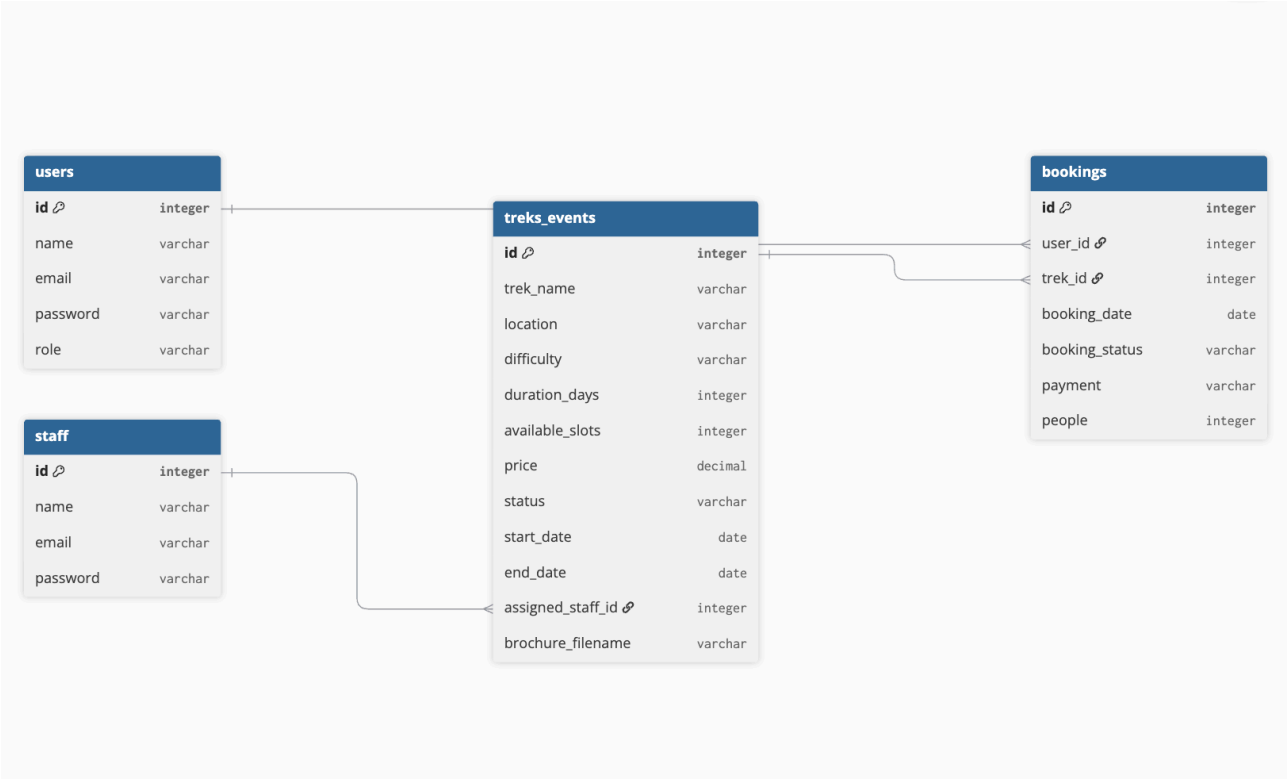
TrekEvent — stores trek information such as id, trek_name, and location.

Booking — stores trek booking details such as id, user_id, trek_id.

Staff — stores staff-related information and their assigned trek details.

**Relationships:**

- One-to-Many → User → Booking
- One-to-Many → TrekEvent → Activity
- One-to-Many → Staff → TrekEvent
- Many-to-One → TrekEvent → Staff




The above image is generated from *dbdiagrom.io* as suggested

**6. API Resource Endpoints:**

| Endpoint | Method | Description |
|---|---|---|
| `/api/treks` | GET | Fetch available trekking events |
| `/api/activities/<id>` | GET | Fetch details of a specific trek |
| `/api/bookings` | GET | Fetch booking information |
| `/api/bookings` | POST | Create a new trek booking |
| `/api/users` | GET | Fetch registered user information |
| `/api/staff` | GET | Fetch staff information |

API routes are implemented using Flask Blueprints in routes/api.py

## 7. Architecture and Features:

**Architecture Overview:**

- **app.py** – Main Flask application entry point and application configuration.
- **config.py** – Application and database configuration.
- **database/** – SQLAlchemy database, migration, and login manager initialization.
- **models/** – Database models for users, staff, treks, and bookings.
- **routes/** – Flask Blueprints handling authentication, admin, staff, user/trek, booking, API, and chart-related functionality.
- **templates/** – Jinja2 HTML templates for Admin, Staff, and User interfaces.
- **static/** – CSS, JavaScript, and application images/backgrounds.
- **instance/** – Local SQLite database storage during development.

**Implemented Features:**

1. User registration and login
2. Daily activity tracking and habit logging
3. Assignment of staff members to treks
4. Trek booking and cancellation by users
5. Available slot management for treks
6. Staff access to assigned trek and participant information

**Additional Features:**

1. Trek brochure PDF support
2. Confirmation before permanently deleting a trek
3. Automatic participant/slot tracking for bookings
4. Visualisation dashboard using Chart.js
5. Custom trekking-themed backgrounds
6. Password visibility toggle on login/registration forms

## 8. Video Presentation

**Drive Link:**

https://drive.google.com/file/d/1u2takJZC5GTIcaYuUj0JB-yqULtZhTpZ/view?usp=sharing